МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №7

з курсу: „Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Метою досліджень по цій темі є процеси побудови прогнозованих текстових даних на основі аналізу вхідних текстових даних з використанням методу ковзного вікна, з врахуванням специфіки побудови мовних моделей для навчання LLM. В наведених завданнях можна використати тексти програмних кодів, створених на попередніх лабораторних заняттях.

Завдання.

1. Познайомитися з методикою обробки вхідних текстових даних «слово по слову» та прогнозування наступних значень слів в текстових документах, які обробляються, в основі якої використовується метод ковзного вікна.

2. Вибрати самостійно кілька простих прикладів текстів для виконання експериментів.

3. Слідуючи пунктам завдання, описаного в цій інструкції, виконати експерименти з обраними прикладами текстів.

4. Представити повний письмовий звіт з виконаних досліджень

Хід роботи

Після вивчення теоретичних основ можна переходити безпосередньо до проведення експериментів.

Використовуючи токенізатор BPE для токенізації казки “The emperor’s new clothes”, виведемо у консоль кількість токенів у тексті та перші 10 з них:

```
import tiktoken

with open("THE EMPEROR’S NEW CLOTHES.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

tokenizer = tiktoken.get_encoding("gpt2")
enc_text = tokenizer.encode(raw_text)

print("Кількість токенів у тексті: ", len(enc_text))
enc_sample = enc_text[50:]

print("Перші 10 токенів обрізаного тексту: ", enc_sample[:10])
```

```
Кількість токенів у тексті:  2609
Перші 10 токенів обрізаного тексту:  [21421, 393, 262, 15505, 11, 2845, 329, 262, 6443, 788]
```

Рис.1 Кількість токенів у тексті та перші 10 з них

Далі, щоб на практиці продемонструвати роботу методу «ковзного вікна», створимо приклад пар «вхід - ціль». Спершу визначимо довжину контексту: у нашому випадку модель отримує чотири попередні токени й на їх основі повинна передбачити наступний. Після цього виконаємо декодування

обох наборів індексів назад у текст, аби побачити, які саме слова відповідають цим токенам. Покажемо, як саме це вікно поступово зміщується вздовж послідовності:

```
context_size = 4
x = enc_sample[:context_size]
y = enc_sample[1:context_size+1]
print(f"x: {x}")
print(f"y: {y}")

print("У декодованому вигляді: ")
print("x: ", tokenizer.decode(x))
print("y: ", tokenizer.decode(y))

print("Метод ковзного вікна: ")
for i in range(1, context_size+1):
    context = enc_sample[:i]
    desired = enc_sample[i]
    print(tokenizer.decode(context), "-", tokenizer.decode([desired]))
```

```
У декодованому вигляді:
x:   theatre or the chase
y:   or the chase,
Метод ковзного вікна:
 theatre - or
 theatre or - the
 theatre or the - chase
 theatre or the chase - ,
```

Рис.2 Результат виконання методу ковзного вікна

Після виконання коду ми отримали числові послідовності токенів: x містить [21421, 393, 262, 15505], а y - [393, 262, 15505, 11]. Декодування цих послідовностей показує, що x відповідає словосполученню «theatre or the chase», а y - «or the chase,».

Метод ковзного вікна демонструє послідовне зміщення контексту для прогнозування наступного слова. У нашому прикладі це відбувається так:

1. «theatre» - передбачає «or»
2. «theatre or» - передбачає «the»
3. «theatre or the» - передбачає «chase»
4. «theatre or the chase» - передбачає «,»

Таким чином формується послідовність пар «вхід-ціль».

Наступним етапом реалізуємо клас GPTDatasetV1, призначений для формування навчального набору даних для мовної моделі з використанням методу ковзного вікна:

```python
import torch
from torch.utils.data import Dataset, DataLoader

class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        token_ids = tokenizer.encode(txt)

        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i + max_length]
            target_chunk = token_ids[i + 1: i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]
```

У конструкторі __init__ текст спочатку конвертується у числову послідовність токенів за допомогою токенізатора. Потім за допомогою циклу формуються пари «вхід–ціль»: змінна input_chunk містить послідовність токенів довжини max_length, а target_chunk - ті самі токени, зміщені на один крок вперед, тобто слова, які модель має передбачити. Параметр stride визначає, на скільки позицій зміщується ковзне вікно при формуванні наступної пари.

Метод __len__ повертає кількість таких вікон у наборі даних, а __getitem__ повертає конкретну пару «вхід–ціль» за заданим індексом. Після виконання цього коду формується об’єкт класу Dataset, який легко інтегрувати з DataLoader у PyTorch.

Після реалізації класу можна перейти до функції create_dataloader_v1, яка формує об’єкт DataLoader для пакетного завантаження даних під час навчання моделі:

```
def create_dataloader_v1(txt, batch_size=4, max_length=256,
                         stride=128, shuffle=True, drop_last=True,
num_workers=0):
    tokenizer = tiktoken.get_encoding("gpt2")

    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        drop_last=drop_last,
        num_workers=num_workers
    )
    return dataloader
```

Після виконання цього коду формується DataLoader, який забезпечує послідовну подачу пакетів токенізованого тексту для тренування моделі або проведення експериментів із методом ковзного вікна.

Наступним кроком можна переходити до тестування різних параметрів цього процесу:

```
print("Тест 1: batch_size=4, max_length=10, stride=5")
dataloader = create_dataloader_v1(text, batch_size=4, max_length=10,
stride=5)
inputs, targets = next(iter(dataloader))
print("Inputs shape:", inputs.shape)
print("Targets shape:", targets.shape)
print("Inputs:", inputs)
print("Targets:", targets)

print("\nТест 2: batch_size=2, max_length=10, stride=2")
dataloader = create_dataloader_v1(text, batch_size=2, max_length=10,
stride=2)
inputs, targets = next(iter(dataloader))
print("Inputs shape:", inputs.shape)
print("Targets shape:", targets.shape)
print("Inputs:", inputs)
print("Targets:", targets)

print("\nТест 3: batch_size=3, max_length=20, stride=10")
dataloader = create_dataloader_v1(text, batch_size=3, max_length=20,
stride=10)
inputs, targets = next(iter(dataloader))
print("Inputs shape:", inputs.shape)
print("Targets shape:", targets.shape)
print("Inputs:", inputs)
print("Targets:", targets)
```

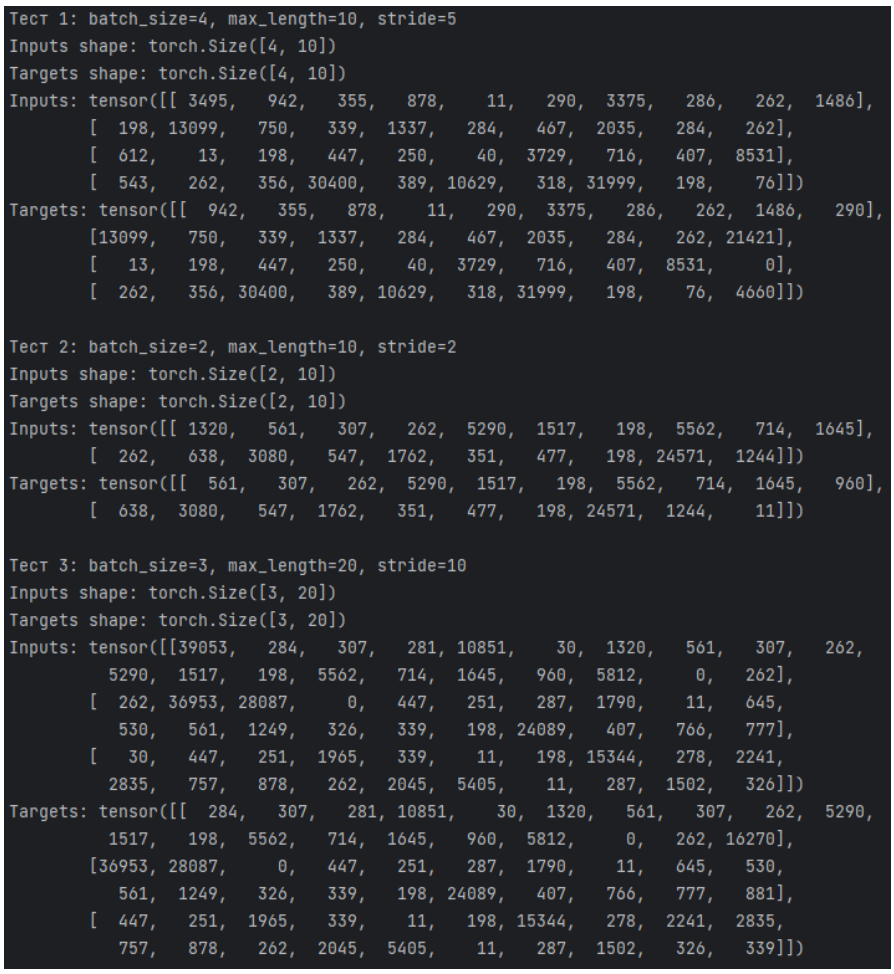


Рис.3 Результати тестування dataloader

У першому тесті з batch size 4, max length 10 і stride 5 вхідні та цільові послідовності мають правильні розміри, а зміщення вікна на половину довжини забезпечує перекриття токенів між сусідніми пакетами. Це дозволяє моделі бачити контекст і підвищує точність передбачення наступного токена.

У другому тесті з batch size 2, max length 10 і stride 2 спостерігається менше перекриття між вікнами, оскільки stride менший. Це призводить до більшої кількості пакетів для навчання і може покращувати здатність моделі до узагальнення, але збільшує обсяг оброблюваних даних.

У третьому тесті з batch size 3, max length 20 і stride 10 вхідні та цільові послідовності довші, що дає моделі більше контексту для передбачення наступного токена. Зі збільшенням довжини вікна та stride модель отримує ширший контекст і може краще навчатися на структурі тексту, але зменшується кількість пакетів у батчі через більший розмір кожного вікна.

Загалом, результати показують, що параметри batch size, max length і stride безпосередньо впливають на кількість пакетів, обсяг контексту та ефективність навчання. Оптимальний вибір цих параметрів залежить від доступної пам’яті та цілей навчання моделі.

```
def show_batch_as_text(inputs, targets, tokenizer, num_examples=3):
    print("\nТекстове представлення кількох тестів\n")

    for i in range(min(num_examples, len(inputs))):
        inp_text = tokenizer.decode(inputs[i].tolist())
        tgt_text = tokenizer.decode(targets[i].tolist())

        print(f"Тест {i+1}:")
        print("Вхід  ->", inp_text)
        print("Ціль  ->", tgt_text)

data_iter = iter(dataloader)
inputs, targets = next(data_iter)

show_batch_as_text(inputs, targets,tokenizer, num_examples=5)
```

```
Тест 1:
Вхід  ->  what they
had begun. However, they put
Ціль  ->  they
had begun. However, they put all
Тест 2:
Вхід  -> The whole city was talking of the splendid cloth which
Ціль  ->  whole city was talking of the splendid cloth which the
```

Рис.4 Тестування декодування токенів

На цьому етапі ми декодуємо токени, отримані з полів inputs і targets у DataLoader, назад у звичайний текст, щоб переконатися, що фрагменти послідовностей формуються правильно. Для цього використовується токенізатор GPT-2, який перетворює числові токени у читабельні слова та символи. Після декодування ми можемо побачити реальні пари «вхід - ціль»: ліворуч подається текст, який модель отримує як контекст, а праворуч - текст, який вона повинна передбачити. Таке представлення дозволяє наочно оцінити, чи коректно працює механізм ковзного вікна та чи формує завантажувач даних осмислені, логічні фрагменти, придатні для навчання мовної моделі. Перегляд таких прикладів допомагає впевнитися, що модель на кожному кроці отримує правильний контекст і правильну наступну частину тексту, яку має передбачити.

Завершальним кроком експерименту стало вивчення того, як параметри max_length і stride впливають на побудову пар «вхід–ціль» у процесі застосування ковзного вікна. З цією метою була розроблена функція test_dataloader_params, яка по черзі запускала завантажувач даних з різними наборами параметрів і демонструвала отримані текстові фрагменти. Такий підхід дав змогу простежити, як зміни конфігурації впливають на структуру сформованих послідовностей. Загалом було виконано п’ять тестів:

```
def test_dataloader_params(raw_text, params_list):
    tokenizer = tiktoken.get_encoding("gpt2")

    for i, params in enumerate(params_list, 1):
        max_length, stride = params

        dataloader = create_dataloader_v1(
            raw_text,
            batch_size=1,
            max_length=max_length,
            stride=stride,
            shuffle=False
        )

        data_iter = iter(dataloader)
        inputs, targets = next(data_iter)

        print(f"\nТест {i}: max_length={max_length}, stride={stride}")
        print("Вхідні токени:", inputs.tolist()[0])
        print("Цільові токени:", targets.tolist()[0])
        print("Вхід:", tokenizer.decode(inputs[0].tolist()))
        print("Ціль:", tokenizer.decode(targets[0].tolist()))

params_list = [
    (2, 2),
    (4, 1),
    (4, 2),
    (8, 2),
    (8, 8)
]

test_dataloader_params(text, params_list)
```

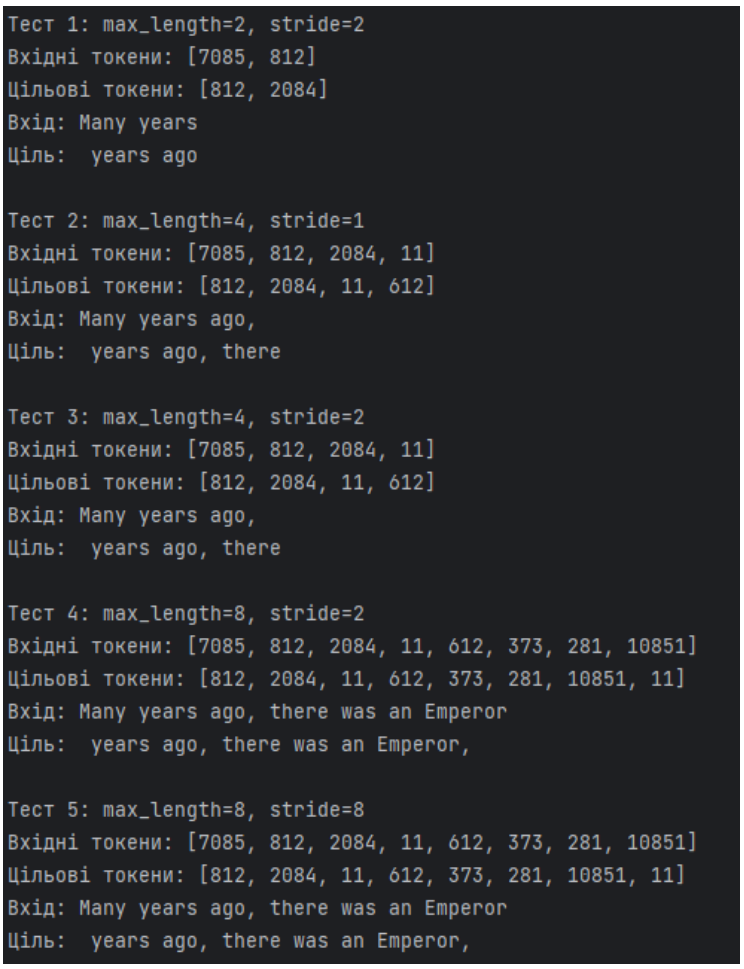


Рис.5 Тестування впливу параметрів на формування пар

Під час перевірки роботи завантажувача даних були проаналізовані результати для різних комбінацій параметрів max_length і stride.

У першому тесті, де розмір вікна дорівнював двом токенам і крок також становив два, модель отримувала мінімальний контекст, а сформовані пари «вхід–ціль» відображали лише найкоротші послідовності.

У другому тесті збільшення max_length до чотирьох та використання stride=1 дало змогу побачити щільне перекриття між вікнами, тому вхідні та цільові фрагменти стали довшими й плавнішими.

Третій тест з такими самими max_length, але stride=2 показав, що зменшення перекриття не впливає на загальну структуру фрагментів, але змінює динаміку пересування вікна.

У четвертому тесті, де вікно було розширене до восьми токенів, послідовності стали значно змістовнішими, що забезпечило кращий контекст для прогнозування.

П’ятий тест з однаковими max_length=8 та stride=8 продемонстрував роботу без перекриття: кожен фрагмент створювався повністю автономно, що чітко видно з відповідних пар «вхід–ціль».

Висновок:

У ході цієї лабораторної роботи було розглянуто принцип дії методу ковзного вікна, що застосовується для формування навчальних даних у задачах тренування мовних моделей. На основі класу GPTDatasetV1 та функції create_dataloader_v1 було реалізовано генерацію пар «вхід–ціль» із різними параметрами довжини контексту та кроку зсуву. Проведені експерименти продемонстрували, що невелике значення stride призводить до значного перекриття сусідніх вікон, завдяки чому фрагменти тексту зберігають сильний зв’язок між собою. Якщо ж stride збільшується, спільних частин між вікнами майже не залишається, і послідовності стають менш взаємопов’язаними. Параметр max_length визначає обсяг контексту, який отримує модель: довше вікно дозволяє охопити більше інформації, але водночас потребує додаткових обчислювальних ресурсів. У результаті виконаної роботи було засвоєно, як саме формується пара «вхід–ціль» для тренування великих мовних моделей, а також досліджено, як ключові параметри ковзного вікна впливають на побудову навчальних прикладів.